بسم الله الرحمن الرحيم

شرح قصير وشامل للكورس السادس

فهرس الكورس

الأساسيات

- Short Hand IF
- Ranged Loop
- Validate Number
- Bitwise AND
- Bitwise OR

الدوال و المفاهيم

- Declaration vs Definition
- Default Parameters
- Function Overloading

- Call Stack
- Recursion
- Static Variables
- Auto Variables
- Register Variable

printf & Formatting

- Integer Format
- Float Format
- String Format
- Setw Manipulator

Arrays & Vectors

- 2D Arrays
- Vector Intro
- Adding & Removing
- Vector of Struct
- Vector Functions
- References

Pointers & Memory

- Pointer Intro
- Dereferencing
- Pointer vs Reference
- Call by Pointer
- Pointer & Arrays
- Pointer & Struct
- Void Pointer
- Dynamic Memory
- Dynamic Arrays
- Stack vs Heap

Vectors Part II

- Access Elements
- Change Elements
- Iterators
- Try & Catch
- String Methods

#9 - Short Hand IF

ال `Short Hand IF` هي العملية الثلاثية الوحيدة في اللغة (Ternary Operator).

و هي عبارة عن اختصار سريع وبسيط لعمليات ال If Statment ذات السطر الواحد

يتم كتابتها كالتالي:

```
Condition ? expression1 : expression2 ;
```

تتحقق `expression 1` اذا كانت ال `Condition` صحيحة

و تتحقق `expression 2` اذا كانت ال `Condition` خاطئة

انظر المثال :

```
int main(){
    int Mark = 90;
    string result;
    if (Mark >= 50) {
        result = "Pass";
    }
    else {
        result = "Fail";
    }
}
```

انظر كيف يمكننا استبدال `If / else` بسطر واحد :

```
int main(){
    int Mark = 90;
    string result;
    result = (Mark >= 50) ? "PASS" : "FAIL";
}
```

وبالطبع في مكان الPass والFail يمكن استخدام function ترجع قيمة ما أو اي عملية رياضية

المهم ان لو الشرط صحيح هترجع القيمة الاولى ولو الشرط خاطئ هترجع القيمة الثانية

كذلك يمكن بسهولة استخدامها أكثر من مرة في سطر واحد لدعم عمليات - If - else if - else الثلاثة

```
int main(){
    int Num = 1;
    string result;
    result = (Num == 0) ? "Zero" : (/*مرة أخرى*/ (Num > 0) ?
"Positive" : "Negative");
}
```

#10 - Ranged Loop

يوجد نوع اخر من ال Loops وهو الRanged Loop وهو نوع يسمحلك بالمرور علي عناصر أيArray بسهولة فائقة

Ranged Loop Syntax

```
for (datatype variable : collection)
{
    // code here
}
```

datatype → نوع البيانات (int, string...)

variable → متغير يمثل كل عنصر

collection → (array) - (vector) - (container) أي

مثال بسيط :

بدل كتابة الكود بهذا الشكل

```
for (int i = 0; i < 5; i++){
    cout << arr[i] << endl;
}
```

(لاحظ انك مضطر تكون عارف طول الأراي), تقدر تكتبه بهذا الشكل

```
int arr[] = {10, 20, 30, 40, 50};
for (int num : arr) {
    cout << num << endl;
}
```

الكود بيسمحلك بسهولة تمشي علي العناصر و تقراهم أو حتى تعدل عليهم من غير ما كنت بتتعب في ال For loop العادية

ازاي ممكن أعدل عليهم ؟؟

تلقائيا , الكود فوق يمكنه القرائة فقط و اذا عدلت في المتغير num لن يقوم بتعديل عناصر الأراي

و لكن ببساطة يمكنك استخدام & و التمرير بالمرجع :

```
int arr[] = {1,2,3};

for (int &x : arr) {
    x = 100;
}
```

و يمكن ايضا استخدامه مع ال vectors (نوع أخر يشبه ال Array)

```
#include <vector>
using namespace std;

vector<int> v = {5, 7, 9};

for (int x : v) {
    cout << x << endl;
}
```

لاحظ : لا يمكنك استخدام Ranged loops علي الArray اذا كانت ممررة داخل ال Function

و لكن يمكن فعل ذلك مع ال Vectors , لاحظ التالي :

```
void Print(int arr[], int len)
{
    for (int x : arr)   //  هذا لن يعمل
    {
        cout << x;
    }
}

void Print(vector<int> v)
{
    for (int x : v)
    {
        cout << x << endl; // يعمل بدون أي مشاكل
    }
}
```

متشغلش بالك بالفيكتورز دلوقتي هناخدها بعدين

#11 Validate Number

اذا طلبنا من المستخدم ادخال رقم و قام بادخال حروف بدل ارقام بيحصل مشاكل في الكود و بيعلق

الحل اننا نستخدم `cin.fail()` اللي بتعرفنا اذا كان المستخدم أدخل مدخلات مناسبة أم لا

بنضيف كمان السطرين دول , و اللي بيمسحو المدخلات الخطأ اللي مسببة كراش في ال cin و يسمحلها تبدأ تشتغل من جديد :

```
cin.clear();
```

```
cin.ignore(std::numeric_limits<std:: streamsize>::max(),
'\n');
```

و الكود بيكون كالتالي :

```
int ReadNumber(){
    int Number;
    cout << "Please enter a number?" << endl;
    cin >> Number;
    while (cin.fail()){
        // if user didn't input a number
        cin.clear();
        cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
        cout << "Invalid Number, Enter a valid one:" << endl;
        cin >> Number;
    }
    return Number;
}
```

ده بيجبر المدخلات تكون صحيحة دايما

#12 Bitwise AND operator

كلنا عارفين ال `AND` العادية اللي هي `&&` بس دي اسمها Logical AND

انما اللي هناخدها دلوقتي هي ال `Bitwise AND` بعلامة واحدة فقط `&`

بتعمل ايه؟

هي فعليا عملية بين الارقام بتقوم علي تحويل الرقمين للنظام الثنائي و مقارنة كل بت مع البت المقابل له

ازاي هيقارن البتات ببعضها ؟ , اعتبر ان 1 هي true و false هي 0

مثال : (12 & 25) ايه النتيجة ؟

`00011001 = 25` `00001100 = 12`

```
    00001100
    00011001 &
    --------
    00001000 = 8 (decimal)

12 & 25 = 8
```

#13 Bitwise OR operator

نفس الموضوع لكن مع ال OR و تكتب | واحدة بدل ||

مثال : (12 | 25) تساوى ؟؟

```
    00001100
    00011001 |
    --------
    00011101 = 29 (decimal)

12 & 25 = 29
```

العملية	الرمز	الوظيفة	مثال (12 و 25)
Bitwise AND	&	لازم "الاثنين" يكونوا 1 عشان تطلع 1	8
Bitwise OR	|	لو "واحد بس" بـ 1 تطلع 1	29

الدروس من 14 ل 19 هي اضافات علي ال Functions

Declaration Vs - #14 Definition

عند تعريف متغير `int x;` بدون اسناد قيمة هذا يسمى `Declaration` و عند اسناد قيمة له في اي وقت هذا يسمى `Definition`

تماما الوضع مع الدوال , بإمكانك تعريف رأس الدالة بمفرده :

```
Type FunctionName (Parameters);
```

كأنك بتعرف رأس الدالة فقط ثم تضع ;

و من ثَم يمكنك إسناد التعاليم البرمجية خاصتها في أي مكان أخر في الكود

و هذا يمكّنك من استخدام الدالة في اي مكان في الكود حتى قبل الإسناد

لاحظ ان في الاصل انت مجبر تعرف الدالة قبل ما تستدعيها بس دلوقتي كل ما عليك انك تعمل Declaration و ساعتها تقدر تعرفها بعدين في اي مكان في الكود :

```
#include <iostream>
using namespace std;
// function declaration
void add(int, int);

int main() {
    add(10, 20);
    return 0;
}
// function Definition
void add(int a, int b) {
    cout << (a + b);
}
```

شوفت ان الكود شغال طبيعي حتى مع ان استدعاء دالة add حدث قبل تعريفها اصلا , و ده بسبب ال function declaration

#15 - Default Parameters

عند وضع باراميتر في اي دالة من الضروري ادخال له قيمة عند استدعاء الدالة
و لكن
عند استخدام خاصية ال Default Value يقوم المترجم تلقائيا بوضع هذه القيمة في المتغير في حال اذا لم يتم تمرير قيمة

يعني : هذه الخاصية عبارة عن قيمة بديلة توضع في الباراميتر اذا لم يتم تمرير قيمة إليه

كيف يكتب؟

```
returnType FunctionName(type param = default_value);
```

```
#include <iostream>
using namespace std;

void PrintMessage(string name = "Guest") {
    cout << "Hello " << name << endl;
}

int main() {
    PrintMessage("Ali");
    PrintMessage();
}
```

الكود أعلاه يعبر عن دخول مستخدم للسستم , اذا وضع اسمه فأهلا و سهلا و لكن اذا لم يضع فلن نترك النظام يرفع Error و لكن يكون اسمه Guest كبديل

لاحظ : من الاجباري وضع ال Default Parameters في نهاية البارامیترات

```
#include <iostream>
using namespace std;
int MySum(int a, int b, int c = 0, int d = 0) {
    return (a + b + c + d);
}
int main() {
    cout << MySum(10, 20) << endl;
    cout << MySum(10, 20, 30) << endl;
    cout << MySum(10, 20, 30, 40) << endl;
    return 0;
}
```

الكود أعلاه يوضح إختيارية وضع قيمة للDefault Parameters و إجبارية وضع قيمة لباقي البارامترات

و كما قلت يجب وضع Default Parameters في النهاية

```
int MySum(int a = 0, int b = 0, int c, int d); // -> Error
```

طيب ايه فايدتها؟

بدل ما تعمل 5 دوال overload :

```
void Print();
void Print(string name);
void Print(string name, int age);
```

تعمل واحدة فقط :

```
void Print(string name="Guest", int age=0);
```

طب ايه هو اصلا ال Overloading

#16 - Function Overloading

تخيل ان عندك دالة اسمها `Print` عايزها مرة تطبع int و مرة تطبع String و مرة تطبع Array و هكذا ,, هتعمل ايه؟

غالبا هتعرف تلات دوال

```
PrintInt (int number);
PrintString (string name);
PrintArray (int Array[]);
```

هو حل صحيح و لكن في فكرة انت مش مدركها : و هي ان انت اساسا مش مجبر تغير اسمها كل مرة

عادي جدا انك تسميهم كدا :

```
Print (int number );
Print (string name);
Print (int Array[]);
```

و ده بيسهل كتير جدا سواء في وضوح الكود أو تنظيمه

طيب هو ده ميعتبرش اعادة تعريف و يؤدي ل Erorr ؟
الاجابة لا

ده طريقة ذكية لجعل الدالة الواحدة متعددة المهام , حتى cout نفسها فيها داخلي OverLoading

المترجم بيفهم لوحده انت عايز انهي تعريف بناءً علي مدخلات الدالة

طب ايه طرق اعادة التعريف المسموحة و ايه غير المسموحة؟

الطرق المسموحة :

اختلاف النوع

```
void print(int x);
void print(string x);
```

اختلاف العدد

```
void print(int x);
void print(int x, int y);
```

اختلاف الترتيب

```
void show(int x, double y);
void show(double y, int x);
```

الطرق الغير المسموحة :

تغيير نوع ال return فقط

```
int sum(int a, int b);
double sum(int a, int b); // خطأ
```

مثال الطباعة :

```
void print(int x)
    { cout << "number: " << x << endl; }
void print(string text)
    { cout << "text: " << text << endl; }
void print(double x)
    { cout << "double: " << x << endl; }
int main(){
    print(10);
    print("Hello");
    print(3.14);
}
```

#17 - Call Stack / Call Hierarchy

ال Call Stack هو مكان في الذاكرة بيتحفظ فيه ترتيب الدوال اللي اتناديت أثناء تنفيذ البرنامج.

كل ما دالة تتنادى بتتحط فوق

و كل ما دالة بتنتهي بتتشال من فوق

```
#include <iostream>
using namespace std;

void FunctionC() {
    cout << "Hi Im inside  C\n";
}
void FunctionB() {
    FunctionC();
    cout << "Hi Im inside  B\n";
}
void FunctionA() {
    FunctionB();
    cout << "Hi Im inside A\n";
}
int main() {
    FunctionA();
    cout << "Hi Im in the main Function\n";
}
```

ايه اللي بيحصل بالترتيب؟

1- الدالة main اشتغلت , حطها في الاستاك

Active Frame

main

2- تم استدعاء الدالة A :

Active Frame

FunctionA

↑

main

(دلوقتي الmain هتفضل مستنية A تخلص شغلها عشان تكمل.)

3- الدالة A استدعت الدالة B :

Active Frame

FunctionB

↑

FunctionA

↑

main

(الدالة A هتفضل مستنية B تخلص)

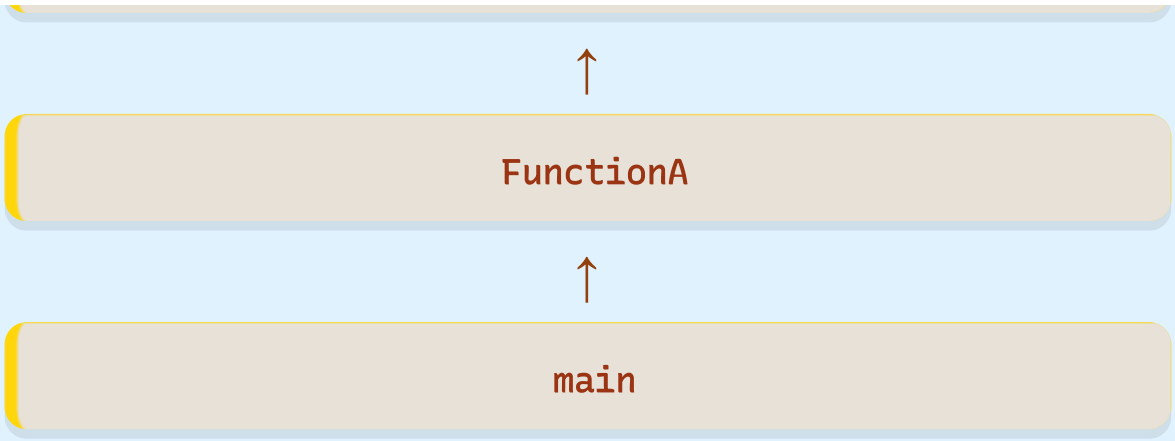
4- الدالة B استدعت الدالة C :

Active Frame

FunctionC

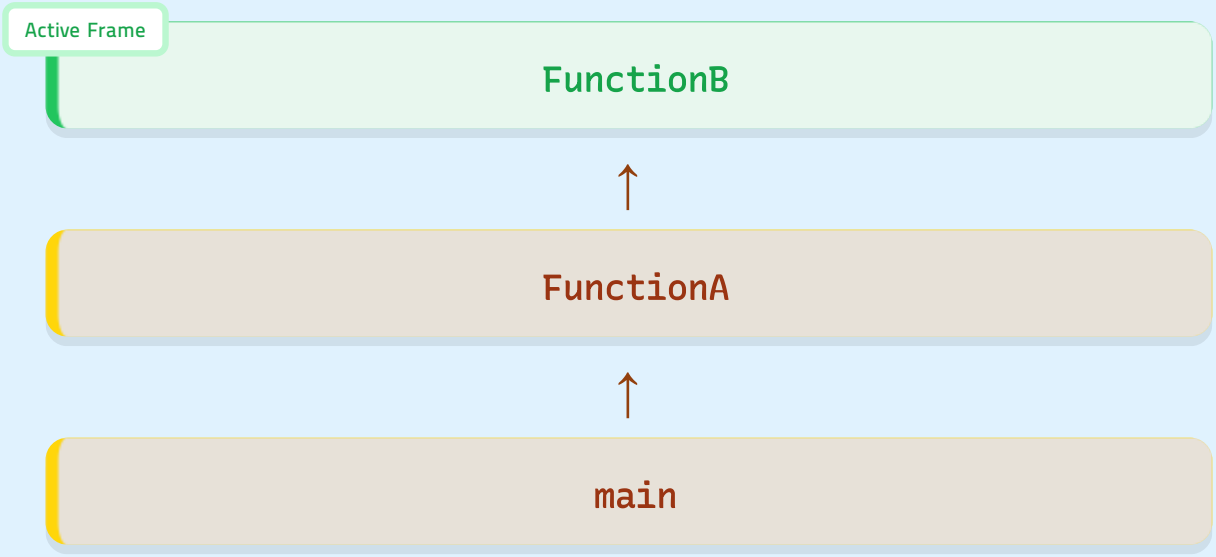
↑

FunctionB



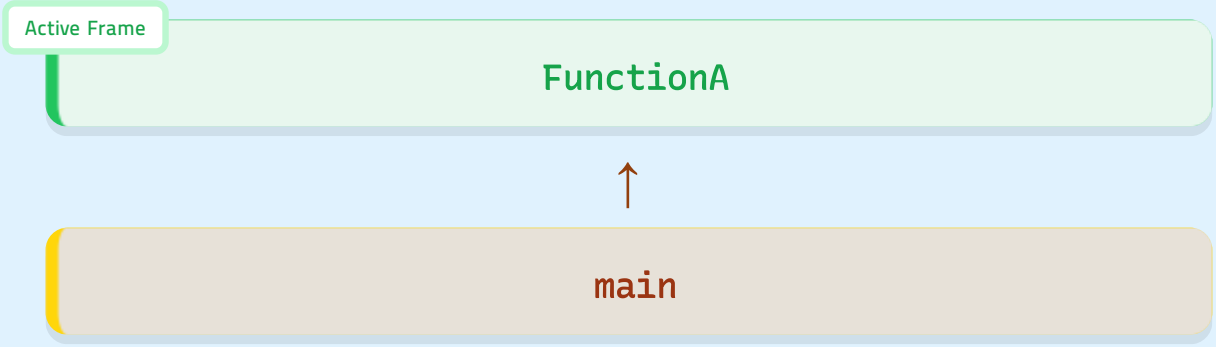
5- و بعدها الدالة C اشتغلت و أدت خدمتها , بالتالي المترجم هيعمل ايه؟

هيشيلها و يبص علي الاستاك -> لقى الدالة B فا يقوم رايحلها و يكمل فيها

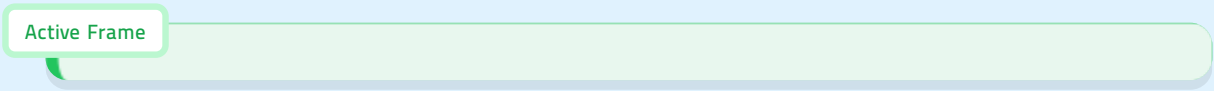


6- الدالة B خلصت ,, نعمل ايه؟

المترجم هيروح الاستاك يشيلها و هيلاقي تحتها A



وهكذا



اللوون الاخضر بيعبر عن الدالة اللي شغالة دلوقتي

و ساعتها الناتج هيكون :

```
Hi Im inside  C
Hi Im inside  B
Hi Im inside  A
Hi Im in the main Function
```

ده بيكون اسمه `LIFO` يعني `Last In -> Frist Out` (أخر واحد دخل هو اول واحد يطلع)

و ايه لازمة كل ده ؟

ببساطة بدون ال Call Stack المترجم مش هيكون عارف يروح فين من بعد ما خلص Function C

و الاهم من ذلك ان ده اساس مفهوم ال `Recursion` ان الدالة تستدعي نفسها بتكرار , هناخده في الدرس 19

يعني دلوقتي اللي بيحصل بالترتيب :

1- الدالة تستدعي دالة تانية

2- الدالة الجديدة تتحط فوقها في الستاك

3- الدالة اللي فوق لما تخلص بيتعملها حذف POP

4- الاستاك يرجع يبص علي اخر دالة (اللي كانت قبلها) و يكمل فيها

الدرس 18 كان شوية Tips علي الVS

recursion - #19

احنا شفنا ازاي الدوال بتنادي بعضها بس ماذا لو كانت الدالة بتنادي نفسها؟

ال `recursion` هو ان الدالة تنادي نفسها عشان تحل مشكلة كبيرة بتقسيمها لمشاكل صغيرة قابلة للحل بسهولة

مثلا عندك مشكلة : انا عايز اجيب فاكتوريال ال 10 يعني عايز اجيب 10 * 9 * 8 * 7 .... * 2 * 1

ازاي نقدر نقسم المشكلة دي؟

أولا هنقول ان `10! = 10 * 9!`

يعني مضروب ال10 يساوي 10 في مضروب ال9

ادينا حلينا اول خطوة , نروح نخلي الدالة تنادي نفسها و تجيبلنا مضروب ال9

تقوم الدالة تفضل تكرر نفس العملية تاني و تالت ... لحد ما توصل لحاجة مهمة اسمها الBase Case (نقطة التوقف)

نقطة التوقف في حالتنا هي ال1 لأن مضروب الواحد معروف قيمته ب 1 و مفيش ارقام من تحته , بالتالي هنقول للبرنامج لما توصل لل1 خلاص افراج , رجع القيمة بتاعتك

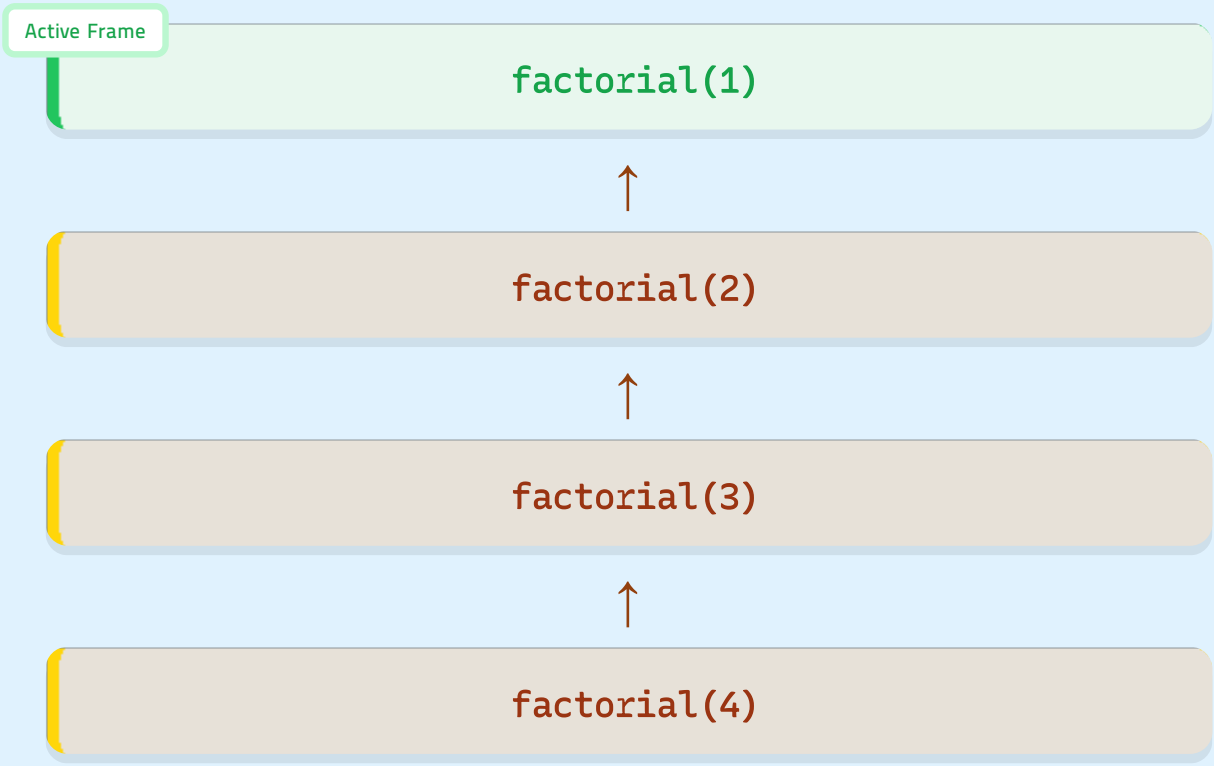
دلوقتي نقدر نشوف المثال بالكود :

```
#include <iostream>
using namespace std;
int factorial(int n){
    if (n == 1)    // base case (نقطة التوقف)
        return 1;

    return n * factorial(n - 1); // recursive call
    //      (n! = n * (n-1)!)
}
int main(){
    cout << factorial(4);
}
```

ايه اللي حصل بالظبط في الكود ؟

```
4! = 4 × 3!
3! = 3 × 2!
2! = 2 × 1!
1! = 1    ← دي base case
```



ال `factorial(1)` رجعت 1

قامت `factorial(2)` ضربتها في 2 و رجعت 2

قامت `factorial(3)` ضربتها في 3 و رجعت 6

قامت `factorial(4)` ضربتها في 4 و رجعت 24

و هي دي اللاجابة 24

أي ريكيرشن عشان يشتغل صح لازم بالضرورة يحتوي علي حاجتين :

1- Base Case

بدونها البرنامج هيفضل يشتغل للأبد و يحصل Stack OverFlow

2- Recursive Call

أمر الاستدعاء , و يجب بالضرورة يكون بيقلص المشكلة بتاعتنا

أمتى يكون احسنلي استخدمه و امتى ابعد عنه؟

الريكيرشن يستحسن استخدامه في الحالات اللي انت متقدرش تحل فيها المشكلة بال loops

مثلا شوف الكود التالي , بيطبع الارقام من Start ل End :

```
#include <iostream>
using namespace std;
void PrintNumbersFromNtoM(int Start, int End)
{
    if (Start <= End)
    {
        cout << Start << endl;
        PrintNumbersFromNtoM(Start + 1, End);
    }
}
int main() {
    PrintNumbersFromNtoM(1, 10);
    return 0;
}
```

الكود تمام و زي الفل بس في مشكلتين :

1- الاولى اني اقدر بسهولة اني احل المشكلة دي بالloops و هتكون اوضح بكتير

2- التانية ان في الحالات الضخمة بيحصل Stack OverFlow انما ال Loops بتقدر تهندل الحالات الضخمة عادي

و بالمناسبة الloops اسرع في الحالة دي و مش بتاخد مومري زي الستاك

و لكن الريكيرشن في الاصل يستخدم في حل بعض المشاكل الصعبة اللي مش بسيطة لدرجة انها تتحل بالloops

فا متستخدمهاش في الفاضي و المليان انت غالبا مش هتحتاجها علي وجه الخصوص الا في حالات معقدة

#20 - Static Variables

احنا عارفين ان المتغير العادي في اي دالة بيتولد و يموت في نفس الدالة و بيتحذف من الذاكرة لما الدالة تخلص

لكن ال `Static Variable` بيتولد مرة واحدة و بيفضل عايش لحد ما البرنامج كله ينتهي , حتى لو الدالة اللي اتولد فيها انتهت

يعني ببساطة :

1- بيتنشئ مرة واحدة فقط

2- بيفضل يحتفظ بقيمته بين استدعاءات الدالة

3- لا يتم حذفه عند انتهاء الدالة , عمره = عمر البرنامج كله

لكن!

نطاقه (scope) يفضل داخل الدالة فقط. حتى مع انه عايش و الدالة مش شغالة بس مفيش دالة ليها Access عليه الا الدالة اللي اتولد فيها

مثال علي المتغير الطبيعي :

```
void counter() {
    int x = 0;
    x++;
    cout << x << endl;
}
int main() {
    counter();
    counter();
    counter();
}
```

```
1
1
1
```

مثال علي المتغير الاستاتيك :

```
#include <iostream>
using namespace std;
void counter() {
    static int x = 0;
    x++;
    cout << x << endl;
}
int main() {
    counter();
    counter();
    counter();
}
```

```
1
2
3
```

ايه اللي بيحصل في الذاكرة ؟؟

المتغير x لم يتم تخزينه في الاستاك كالعادة , ولكن بيتخزن في `data segment` (جزء ثابت من الذاكرة)

يعني اقرب لكونه Global لكن داخل الدالة اللي اتعرف فيها

ليه نستخدم static أصلاً؟

1- الاحتفاظ بقيمى مشتركة بين استدعاءات الدالة : زي عدد الاستدعاءات أو أخر نتيجة طلعت من الدالة دي

2- تقليل استخدام global variables

3- تحسين الاداء أحيانا , لأن المتغير بيتنشئ مرة واحدة فقط مش كل ما الدالة تشتغل

ملحوظة : لو عملت Static Variable خارج الدالة (يعني مش داخل اي دالة) هيشتغل بالظبط كأنه Global بس مع اختلاف أنه مرئي داخل الملف فقط ولا يمكن الوصول اليه من باقي الملفات

#21 - Automatic Variables

هي طريقة لجعل المترجم يدي المتغير نوعه تلقائيا

يعني بدل ما تحدد النوع كل مرة

```
int x = 10;
double y = 3.14;
string name = "Ali";
```

المترجم بيفهم ان القيمة دي نوعها كذا و يديها للمتغير بتلقائية

```
auto x = 10;
auto y = 3.14;
auto name = "Ali";
```

و ده باستخدام الKeyWord `auto`

طبعا لازم تديلو قيمة

```
auto x; //Error
```

ليه نستخدمه؟

مستقبلا هيكون في بعض الحالات اللي الtype فيها هيكون معقد و طويل فا بنستخدم بداله auto مع ذلك ده غير مستحب برضو

بيكون مفيد برضو لما تستخدمه في الRanged Loops

#22 - Register Variable

الCPU فيه ذاكرة صغيرة جدا بس سريعة جدا اسمها الRegister والرام مقارنة بيها كان بطيء شوية

فا المبرمجين ايام زمان كانو يقولو ليه منقولش للمترجم يحط المتغير ده في الريجيستر بدل الرام عشان السرعة , و ده فعلا حصل و عملو امر اسمه register بيتحط قبل نوع المتغير اثناء تعريفه

```
for (register int i = 0; i < 1000000; i++) {
    // عمليات سريعة
}
```

و كان فعلا بيخليه أسرع في الوصول اليه

بس ده كان زمان انما دلوقتي المترجمات بقت اذكى و بقت بتعمل كده بشكل تلقائي , كل ما تلاقي متغير كثير الاستخدام و كمان مساحة فاضية في الريجيستر كانت بتحطه فيه علي طول لوحدها

فا معتش حد بيستخدم الامر ده عموما

printf

#23 - Integer Format

اللغة فيها دالة مهمة جدا اسمها `printf` الدالة دي تُستخدم لعرض البيانات على الشاشة بطريقة منظمة يمكن التحكم فيها بدقة.

يعني بتنسق طريقة و مكان عرض البيانات اللي عايز تطبعها

و الدرس ده عن طباعة الIntegers

لما تيجي تستخدمها عشان تطبع integers محتاج تديها امر يعرفها ان الرقم الجاي ده عدد صحيح و تعامله معاملة الاعداد الصحيحة

و ده بيتم باستخدام `%d` او `%i`

مثال بسيط :

```
int main() {
    int x = 25;
    printf("%d", x);
}
```

```
25
```

و تقدر تستخدم أكتر من متغير

```
int a = 10;
int b = 20;

printf("%d %d %d", a, b, a + b);
```

```
10 20 30
```

دلوقتي نبدأ نعمل formatting بجد

اولا تحديد عرض الرقم

```
printf("%5d", 42);
```

```
   42
```

المترجم حط خمس مسافات قبل الرقم عشان كتبنا `%5d`

طيب لو انا عايز احط حاجة غير المسافات؟

```
printf("%05d", 42);
```

```
0000042
```

ده بيمليها بأصفار عشان حطينا صفر بعد ال%

ثانيا الطباعة بمحاذاة اليسار

يعني بدل ما يحط المسافات علي الشمال يحطها علي اليمين

```
printf("%-5d Is my age", 18);
```

```
18     is my age
```

مثال كامل

```
int main() {
    int age = 21;
    int score = 7;

    printf("Age: %d\n", age);
    printf("Score: %05d\n", score);
    printf("Score aligned: %-5dEND", score);
}
```

```
Age: 21
Score: 00007
Score aligned: 7    END
```

#24 - Float Format

بيتم تنسيق الارقام العشرية باستخدام `%f`

أولا : الصيغة العادية `%f`

```
float pi = 3.14159;
printf("pi is %f", pi);
```

```
pi is 3.141590
```

لاحظ انه في الصيغة العادية بيطبع 6 ارقام عشرية تلقائيا

ثانيا : التحكم في عدد الأرقام بعد العلامة

يتم بالصيغة `%.(number)f`

```
float x = 3.1415926535;
printf("%.2f", x);
printf("%.4f", x);
```

```
3.14
3.1415
```

ثالثا : الشكل العلمي `%e`

لما الرقم يكون كبير جدا أو صغير جدا

```
long double x = 88888888;
printf("%e", x);
```

```
8.888889e+07
```

يمكن كذلك تنسيق العلامة العشرية أيضا

```
long double x = pow(2, 64);
printf("%.2e", x);
```

```
1.84e+19
```

و هذا يعني : `2^64 = 1.84 × 10^19`

#25 - String Format

هنا بنستخدم `%s`

بس للأسف دالة الprintf واللي هي جاية من لغة C في الاساس , مش بتفهم يعني ايه اوبجيكت اسمه string بس بتفهم الأراي المكونة من الاحرف

يعني احنا بدل ما نمرر string مباشرة بيكون عندك حلين

أولا انك تستخدم char Array بدل string

```
char name[] = "Ahmed";
printf("Hi %s", name);
```

```
Hi Ahmed
```

الحل التاني انك تستخدم string عادي ولكن تستخدم معاه دالة c_str

```
string name = "Abdelrahman";
printf("%s\n", name.c_str());
```

```
Abdelrahman
```

وتقدر كمان تستخدم الطباعة بالمحاذاة زي ما عملنا في الinteger

#26 - Setw Manipulator

هنا انت مش بتطبع عدد مسافات وبعدين تكتب بعدهم

هنا انت "بتحجز" مسافات و بتكتب عليها

يعني مثلا تخيل 8 مسافات نكررهم تلات مرات و نخليهم نجوم عشان يكونو واضحين

```
********

********

********
```

عايز اكتب مرة Ahmed ومرة Mohamed ومرة Abdo

```
***Ahmed

*Mohamed

****Abdo
```

المسافات بتنسق نفسها و بتقص الجزء اللي بتكون محتاج تطبع فيه عشان تعمل محاذاة جميلة

```
cout << setw(10) << "Name"
     << setw(10) << "Age"
     << setw(10) << "Score" << endl;

cout << setw(10) << "Ali"
     << setw(10) << 20
     << setw(10) << 95 << endl;
```

الناتج :

```
      Name        Age      Score
       Ali         20         95
```

طب لو انا عايز اطبع خط طويل بال= مثلا

بتستخدم setfill اللي بتغير المسافات للعلامة اللي انت عايزها

```
cout << setfill('=') << setw(50) << "" << endl;
cout << setfill(' ');
cout << setw(10) << "Name"
     << setw(10) << "Age"
     << setw(10) << "Score" << endl;

cout << setw(10) << "Ali"
     << setw(10) << 20
     << setw(10) << 95 << endl;

cout << setfill('-') << setw(50) << "" << endl;
```

الناتج :

```
==================================
      Name        Age      Score
       Ali         20         95
----------------------------------
```

لاحظ اني رجعتها لطبيعتها تاني باستخدام السطر `cout << setfill(' ');`

#27 - 2D Arrays (Matrix)

المصفوفة ثنائية البعد (2D array) هي ببساطة عبارة عن مصفوفة تقوم بتخزين مصفوفات اخرى داخلها , فبذلك تمثل جدول من الصفوف و الاعمدة

أي أنها تمثل بعدين : البعد الاول هو الصفوف (أفقي) ثم الأعمدة (رأسي)

يتم تعريفها كالتالي :

```
dataType name[rows][columns];
```

```
int arr[3][4];
```

يتم الوصول للعنصر داخلها عن طريق الترتيب الصف (بدايتا من الصفر) ثم رقم العمود (بدايتا من الصفر)

```
arr[0][0]  arr[0][1]  arr[0][2]  arr[0][3]
arr[1][0]  arr[1][1]  arr[1][2]  arr[1][3]
arr[2][0]  arr[2][1]  arr[2][2]  arr[2][3]
```

ثانيا : ادخال القيم

ادخال يدوي عند التعريف

```
int arr[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

تقدر كمان متقسمهاش و تحط كل العناصر جمب بعضها (غير مستحب)

```
int arr[2][3] = {1, 2, 3, 4, 5, 6};
```

اللغة ستملأ الصفين بتلقائية

الوصول لعنصر أو تعديله :

بما ان الترتيب يبدا من الصفر , العنصر `arr[1][2]` مثلا هيكون العنصر التالت في الصف التاني

طباعة المصفوفة

للوصول لعناصر المصوفة فغالبا بنستخدم Nested loops

```
for(int i=0; i<2; i++) {
    for(int j=0; j<3; j++) {
        cout << setw(2) << arr[i][j];
    }
    cout << endl;
}
```

الناتج هيكون

```
 1 2 3
 4 5 6
```

طيب لو عايزين نعرف كل عنصر بPattren معينة

مثلا : كل عنصر يساوي حاصل ضرب الصف في العمود

```
int arr[6][6];
for(int i=0; i < 6 ; i++) {
    for(int j=0; j < 6 ; j++) {
        arr[i][j] = (i + 1) * (j + 1);
    }
    cout << endl;
}
for(int i=0; i < 6; i++) {
    for(int j=0; j < 6; j++) {
        cout << setw(3) << arr[i][j] << " ";
    }
    cout << endl;
}
```

المصفوفة عند طباعتها هتكون بالشكل التالي :

```
  1   2   3   4   5   6
  2   4   6   8  10  12
  3   6   9  12  15  18
  4   8  12  16  20  24
  5  10  15  20  25  30
  6  12  18  24  30  36
```

Vectors

#28 - Intoduction

الVector هي ببساطة Array لكن بأسلوب مختلف و ميزات اضافية

الVectors علي عكس الArray قابلة للتمدد و ليس لها حجم ثابت , يتم استدعائها من المكتبة :

```
#include <vector>
```

تم اختراعها لسد عيوب الStatic Array العادية , تقدر دلوقتي تضيف أو تحذف أو توصل لعدد العناصر بسهولة من خلال دوال مدمجة , و ده دايما بيكون الاحسن في حل المشكلات

كيفية التعريف

```
vector < type > Name;
```

```
#include <vector>
using namespace std;

vector<int> numbers;
```

أو :

```
vector<int> numbers = {1, 2, 3, 4};
```

يمكنك المرور علي العناصر باستخدام Ranged loop بسهولة

```
for (int num : numbers){
    cout << " " << num;
}
```

```
1 2 3 4
```

#29,30 - Adding & Removing

تقدر تضيف عناصر في الفيكتور بواسطة دالة فيه اسمها `push_back`

```
vector<int> v;

v.push_back(5);
v.push_back(10);
v.push_back(20);

for(int i=0 ; i < 3 ; i++)
    cout<<v[i]<<" ";
```

```
5 10 20
```

تقدر بسهولة توصل لعدد عناصر الفيكتورز حاليا باستخدام دالة size

الكود ده بياخد عناصر من المستخدم لحد ما يدخل -1

```
vector <int> Numbers;
int x;
cin >> x;
while (x != -1){
    Numbers.push_back(x);
    cin >> x;
}
```

Removing

عشان تحذف اخر عنصر بتستخدم الدالة `pop_back`

```
vector<int> v;

v.push_back(5);
v.push_back(10);
v.push_back(20);
v.push_back(30);
v.push_back(50);

v.pop_back();
v.pop_back();
v.pop_back();
v.pop_back();

for(int num : v)
    cout << v[i] <<" ";
```

```
5
```

تم حذف اخر اربع عناصر

ايه اللي بيحصل بالظبط؟

المترجم يتعامل مع الvector تماما كأنه Stack

لما تضيف عنصر -> يحطه فوق الستاك

لما تحذف عنصر -> يشوف اخر عنصر و يحذفه

هل اقدر اضيف و احذف في مكان معين؟

تقدر

أولا الاضافة باستخدام `insert`

```
v.insert(position , value);
```

بس لاحظ : الpositionهنا مش عبارة عن رقم , لكن عبارة عن مكان في الذاكرة بيتم كتابته بالصيغة دي :

```
v.begin() + index
```

دلوقتي الصيغة النهائية بتكون :

```
v.insert(v.begin() + index , value);
```

مثال :

```
vector<int> vec = {10, 20, 30, 40, 50};

vec.insert(vec.begin() + 2 , 1000); // 2 يعني بتضيف عنصر في الترتيب
for (int num : vec){
    cout << num << " ";
}
```

```
10 20 1000 30 40 50
```

ثانيا الحذف باستخدام `erase`

وله صيغة مشابهة :

```
v.erase(v.begin() + index);
```

مثال :

```
vector<int> vec = {10, 20, 30, 40, 50};

vec.erase(vec.begin() + 2); // 2 يعني بتحذف العنصر في الترتيب
for (int num : vec){
    cout << num << " ";
}
```

```
10 20 40 50
```

طيب هل من الممكن نعملها بنفسنا؟

انا عملتها بنفسي , هحط الكود في الصفحة الجاية لو مهتم تقراه

```cpp
#include <iostream>
#include <cstdio>
#include <vector>
using namespace std;

void MyInsert(vector<int> &vec, int index, int numberToInsert){
    int CountDeleted = vec.size() - index;
    vector<int> vDeleted;

    for (int i = 0; i < CountDeleted; i++){
        int lastElement = vec[vec.size() - 1];
        vDeleted.push_back(lastElement);
        vec.pop_back();
    }
    vec.push_back(numberToInsert);
    for (int i = CountDeleted - 1; i >= 0; i--){
        vec.push_back(vDeleted[i]);
    }
}
void MyErase(vector<int> &vec, int index){
    int CountDeleted = vec.size() - index - 1;
    vector<int> vDeleted;

    for (int i = 0; i < CountDeleted; i++){
        int lastElement = vec[vec.size() - 1];
        vDeleted.push_back(lastElement);
        vec.pop_back();
    }
    vec.pop_back();
    for (int i = CountDeleted - 1; i >= 0; i--){
        vec.push_back(vDeleted[i]);
    }
}
```

#31 - Vector of structure

تقدر تحط اي نوع للvectors و منهم الstructure مثلا

```
#include <vector>
#include <iostream>
using namespace std;
struct stEmployee{
    string FirstName;
    string LastName;
    float Salary;
};
int main() {
    // std::vector<T> vector_name;
    vector <stEmployee> vEmployees;
    stEmployee tempEmployee;
    tempEmployee.FirstName = "Mohammed";
    tempEmployee.LastName = "Abu-Hahdoud";
    tempEmployee.Salary = 5000;

    vEmployees.push_back(tempEmployee);
    tempEmployee.FirstName = "Ali";
    tempEmployee.LastName = "Maher";
    tempEmployee.Salary = 300;

    vEmployees.push_back(tempEmployee);
    tempEmployee.FirstName = "Aya";
    tempEmployee.LastName = "Omran";
    tempEmployee.Salary = 1000;
    vEmployees.push_back(tempEmployee);

    cout << "Employees Vector: \n\n";
    for (stEmployee &Employee : vEmployees) {
    cout << "FirstName: " << Employee.FirstName << endl;
```

```
    cout << "LastName : " << Employee.LastName << endl;
    cout << "Salary : " << Employee.Salary << endl;
    cout << endl;
    }
    cout << endl;
    return 0;
}
```

#32 - Vector Functions

يوجد دوال اخرى مفيدة في التعامل مع الفيكتورز

v.size()

بتجيب عدد العناصر

v.front()

بتجيب أول عنصر `v[0]`

v.back()

بتجيب اخر عنصر `v[v.size() - 1]`

v.empty()

هل الفيكتور فاضي؟ `True` or `False`

v.capacity()

بتجيب سعة التخزين القصوى للvector (مش مهمة اوي)

v.assign()

استبدال كل القيم بقيمة معينة

```
v.assign(5, 7); // -> [7,7,7,7,7]
```

#33,34 - References

الدرس ده كان مراجعة علي الريفرانس و كمان نقطة زيادة

لما كنا بندي قيمة لمتغير ما باستخدام متغير آخر , بيكون ساعتها كل متغير موجود لوحده ملوش دعوة بالتاني

```
int x = 10;
int y = x ;
y++;
cout << x << " " << y;
```

```
10 11
```

لكن انت تقدر تدي المتغير التاني نفس المرجع بتاع المتغير الاولاني , وساعتها الاتنين بيكونو متصلين ولو غيرت واحد التاني بيتغير نفس التغيير

```
int x = 10;
int& y = x;
y++;
x++;
cout << x << " " << y;
```

```
12 12
```

Pointers

#35 - Introduction

بسم الله الرحمن الرحيم

بدايتا, أي برنامج بيتم تشغيله علي الRAM عن طريق حجز مكان للمتغيرات وأي حاجة تحتاج تخزين , بيخزنله مساحة حسب حجم المتغير , مثلا `int x;` بيحجزله 4 بايت في الذاكرة

الكمبيوتر بيقوم بالوصول اليه عن طريق العنوان فقط , عشان كده ممكن تحط عنوان متغير في متغير تاني ولما تغير علي واحد التاني بيتغير

لان اصلا الكمبيوتر شايفهم نفس الحاجة , نفس المرجع

طيب احنا اتعلمنا في اخر درس ازاي نخلي متغيرين ليهم نفس المرجع , طيب ازاي بقا نخزن "مرجع" ذات نفسه داخل متغير

ب استخدام شيء اسمه الpointer

ساعتها بيكون عندك متغير قيمته اساسا هي قيمة المرجع لمتغير تاني و القيمة دي ليست int وليست string وليست دون ذلك , القيمة دي نوعها pointer عموما , أي : قيمة هكسا بتعبر عن عنوان.

لاحظ ان اي متغير له قيمه و له مرجع

يعني متغير الpointer هو كمان له مرجع مختلف

يعني بيخزن مرجع و بيُشار اليه بمرجع

ليه الpointer موجود؟

1- السرعة و الاداء : بدل ما انقل بيانات كاملة بحالها أنقل عنوانها بس

2- البرامج الكبيرة بيكون الاحسن انها تتحكم في الميموري بنفسها , مش كل حاجة تكون اوتوماتيك

3- فوائد اهم بكتير هتعرفها بعدين

مثال لكيفية استخدامه :

```
#include <iostream>
using namespace std;
int main(){
    int x = 10;
    cout << "x value = " << x << endl;
    cout << "x address = "<< & x << endl;
    int * p;
    p = &x;
    cout << "Pointer Value = " << p;
    cout << "Pointer address =" << p;
    cout << endl;
    return 0;
}
```

```
x value = 10
x address = 0x5ffe6c
Pointer Value = 0x5ffe6c
Pointer address = 0x5ffe60
```

لاحظ دلوقتي أهم حاجتين :

أولا قيمة المؤشر `p` هي نفسها قيمة مرجع المتغير `a`

ثانيا مرجع المؤشر `p` مختلف عن المرجع المخزن داخله

التعريف

بيتم تعريفه عن طريق : نوع المتغير اللي هو بيشاور عليه , بعدين نجمة

يعني `int *` معناها : عرفلي متغير بيخزن مرجع متغير integer

يعني `string *` معناها : عرفلي متغير بيخزن مرجع متغير string

وهكذا

#36 - Dereferencing

طيب انا دلوقتي معايا عنوان الحاوية اللي بتحوي المتغير

ازاي استخدم العنوان ده في الوصول للحاوية و اعدل فيها براحتي او اشوف القيمة اللي فيها و هكذا

ده بيتم باستخدام العلامة * ووضعها قبل الpointer

```
int main(){
    int x = 10;
    int* p = &x;

    cout << p << endl;    // عنوان x
    cout << *p << endl;   // قيمة x
}
```

```
0x61ff08    ← عنوان
10          ← القيمة اللي في العنوان
```

كذلك نقدر نغير قيمة المتغير بواسطة المؤشر اللي بيشاور عليه

```
int x = 10;
int* p = &x;
*p = 50;
cout << x;
```

```
50
```

تقدر تغير قيمة x باستخدام قيمة x ذات نفسها عادي , او باستخدام `*p`

ايه الفرق بين التغيير باستخدام x و التغيير باستخدام *p ؟

فعليا مفيش , لان الاتنين بيشاوروا علي نفس الحاوية

اختصارًا

العلامة & توضع قبل المتغير لإستخلاص المرجع
العلامة * توضع قبل المرجع لإستخلاص المتغير

#37, 38 - To follow up

ايه الفرق بين اني اعمل متغير كـ `References` لمتغير تاني (`int&`)
و بين اني اعمل متغير كـ `Pointer` لمتغير تاني (`int*`)
ركز في الكود التالي :

```
int a = 10;
int* p = &a;
cout << p << endl;
cout << *p << endl;
int b = 20;
p = &b;
cout << p << endl;
cout << *p << endl;
```

النتيجة هتكون :

```
0x5ffe5c    مرجع المتغير (a)
10          قيمة المتغير (a)
0x5ffe58    مرجع المتغير (b)
20          قيمة المتغير (b)
```

شوفت ازاي قدرت اغير المرجع اللي متخزن في P

انما بقا بال `Reference` مقدرش اعمل كدا , المتغيرين بيفضلو لازقين في بعض مقدرش افصلهم

كأنهم متغير واحد بس عنده اسمين , شوف الكود :

```
int a = 10;
int &x = a;
cout << &a << endl;
cout << &x << endl;
int b = 20;
&x = b;     // هيعملك ايرور و مش هيشتغل
//x = b; (لاحظ ان ده مجرد تغيير في القيمة)
cout << &b << endl;
cout << &x << endl;
```

#39 Call by Reference: Using pointers

فاكرين لما كنا بنعمل Swap لمتغيرين؟

كنا بنستخدم ال `Call by Reference` عشان نوصل لقيمهم بالطريقة دي:

```
void swap(int &n1, int &n2){
    int temp;
    temp = n1;
    n1 = n2;
    n2 = temp;
}
int main(){
    int a = 1, b = 2;
    cout << "Before swapping" << endl;
    cout << "a = " << a << endl;
    cout << "b = " << b << endl;
    swap(a, b);
    cout << "\nAfter swapping" << endl;
    cout << "a = " << a << endl;
    cout << "b = " << b << endl;
    return 0;
}
```

طب انا ازاي اقدر اعمل نفس الكلام بالPointers ؟؟

أولا بتيجي في البارامتر وتكتب `int *` بدل `int &` كده احنا بنطلب مؤشرات

بس لما تيجي تغير في قيمة المتغير بتحط قبله نجمة , عشان احنا قلنا قبل كده لما تيجي توصل للمتغير بواسطة المؤشر اللي بيشاور عليه بتحط نجمة قبل المؤشر

و اخيرا لما تمرر المتغير مرر مرجعه `&x` مش قيمته `x`

```
void swap(int *n1, int *n2)/* بدل * التغيير الاول & */{
    int temp;
    temp = *n1; // بتحط نجمة قبل المتغير
    *n1 = *n2;
    *n2 = temp;
}
int main(){
    int a = 1, b = 2;
    cout << "Before swapping" << endl;
    cout << "a = " << a << endl;
    cout << "b = " << b << endl;
    swap(&a, &b); // بتحط & قبل المتغير عشان تمرر مرجعه جوا البوينتر
    cout << "\nAfter swapping" << endl;
    cout << "a = " << a << endl;
    cout << "b = " << b << endl;
    return 0;
}
```

```
Before swapping
a = 1
b = 2
After swapping
a = 2
b = 1
```

#40 - Pointers and Arrays

قبل ما نبدا في معلومة مهمة جدا دلوقتي المفروض تعرفها :

ال Array هي فعليا عبارة عن Pointer بحد ذاتها , و ال Pointer ده بيشاور علي مكان أول عنصر من ال Array

نقولها عن طريق الكود احسن :

```
int Arr[3] = {10, 20, 30};
cout << & Arr[0] /* موقع أول عنصر */ << endl;
cout << Arr << endl;
cout << *Arr << endl;
```

```
0x5ffe64
0x5ffe64
10
```

شفت ازاي الاتنين نفس الحاجة

طب ازاي اوصل لباقي العناصر ؟

ببساطة بتزود قيمة i علي ال Arr

```
int Arr[3] = {10, 20, 30};
cout << & Arr[2] /* موقع تالت عنصر */ << endl;
cout << Arr + 2 << endl;
cout << *(Arr + 2) << endl;
```

```
0x5ffe68
0x5ffe68
2
```

طيب نجرب نعمل pointer علي Array

```
int arr[3] = {10,20,30};
int* p = arr;

cout << *p << endl;       // 10
cout << *(p + 1) << endl;  // 20
cout << *(p + 2) << endl;  // 30
```

البوينتر ساعتها مش بيشاور علي الأراي ككل , هو بيشاور برضو علي اول عنصر فقط

عملنا نفس الحركة و زودنا علي قيمة العنوان ذات نفسه عشان نتحرك بين العناصر

لاحظ اننا كتبنا `int* p = arr` بدل `int* p = &arr` عشان احنا قايلين ان arr ذات نفسها عبارة عن عنوان

متنساش انك تقدر تغير قيمة العناصر باستخدام المرجع

```
int Arr[3] = {1, 2, 3};
int* p = Arr;
*(p + 1) = 20;
*(Arr + 2) = 30;
printf ("%d - %d - %d", Arr[0], Arr[1], Arr[2]);
```

```
1 - 20 - 30
```

من الاخر

Arr
عنوان أول عنصر

***Arr**
قيمة أول عنصر ([Arr[0])

(Arr + i)
عنوان العنصر اللي ترتيبه i

***(Arr + i)**
قيمة العنصر اللي ترتيبه i

```
int* p = Arr;
```

p
عنوان أول عنصر

***p**
قيمة أول عنصر

(p + i)
عنوان العنصر اللي ترتيبه i

***(p + i)**
قيمة العنصر اللي ترتيبه i

بالمناسبة تقدر تعمل `p++` عشان تخليها تشاور علي العنصر اللي بعده

بس مشنفعش تكتب `Arr++` عشان عنوانها ثابت انما البوينتر متغير قابل للتعديل

#41 - Pointers and Structure

طبعا انت تقدر تعمل pointer بيشاور علي Structure عادي بس في هنا فرق صغير

وهو انك لما تيجي تعمل `Access` علي متغيرات الستراكتشر , بتحط `->` بدل النقطة :

```
struct stEmplyee{
    string Name;
    float Salary;
};
int main()
{
    stEmplyee Employee1, *ptr;
    Employee1.Name = "Mohammed Abu-Hadhoud";
    Employee1.Salary = 5000;
    cout << Employee1.Name << endl;
    cout << Employee1.Salary << endl;
    ptr = &Employee1;
    cout << "\nUsing Pointer:\n";
    cout << ptr -> Name << endl;
    cout << ptr -> Salary << endl;
    return 0;
}
```

#42 - Pointer to Void

بطبيعة الحال لو عرفت بوينتر من نوع `int*` و حطيت فيه عنوان متغير integer عادي , جيت بعدين حاولت تحط فيه عنوان متغير float هيديك ايرور, لانك معرفه `int*` وليس `float*` , عشان تحل المشكلة دي بتعرفه من نوع void.

```
void* p;
int x = 5;
float y = 0.5;
ptr = &x;
cout << ptr << endl;
ptr = &y;
cout << ptr << endl;
```

بس في مشكلة صغيرة بتظهر و هي ان البوينتر هنا `genaric` نوعه معمم ,و بسبب كده مش بتقدر تطبع القيمة اللي بيشاور عليها

```
cout << *p ; // Error
```

بيتم حل المشكلة دي عن طريق عمل Casting باستخدام دالة `static_cast<type*>` كالتالي :

```
ptr = &y;
cout << ptr << endl;
cout << *(static_cast<float*>(ptr)) << endl;
ptr = &x;
cout << ptr << endl;
cout << *(static_cast<int*>(ptr)) << endl;
```

#43 - Dynamic Memory

احنا في طبيعة الحال لما نعرف متغير بيكون له شوية ضوابط : محطوط في الStack - بيتحذف لما الدالة بتاعته تنتهي

انت مش بتتحكم في الحجز و المسح الكمبيوتر هو اللي بيتحكم فيهم

و لكن باستخدام الPointers نقدر بسهولة نتحكم في الذاكرة براحتنا

هنبدأ باننا نعرف بوينتر طبيعي

```
int* p;
```

بس لما نديله العنوان هنديله المرادي `new int`

```
p = new int;
*p = 10;
```

اللي حصل دلوقتي : تم حجز مكان في الذاكرة

و بعدين أخد عنوان المكان ده حطه جوا البوينتر

المكان ده دلوقتي ملوش قيمة ولا حتى له اسم

و أهم حاجة : مش بيتحذف الا لما انت تأمر بحذفه

بيتخزن في الذاكرة الديناميكية (Dynamic / Heap) , و في طرق تانية لتعريفه :

```
int* p = new int;
int* p = new int (10); // تديه عنوان المكان في الذاكره و قيمته كمان
```

الحذف

الحذف هنا مهم جدا عشان المكان اللي اتحجز في الذاكرة مش مجبر يتحذف لما الدالة بتاعته تخلص و لازم نحذفه بنفسنا

وده بيتم باستخدام الكلمة المفتاحية `delete`

```
delete p;
```

ده مفيد في ان لو البرنامج هيفضل شغال لفترة طويلة و انت معرف متغيرات كتير مش هتستخدمها في الوقت الحالي فانت بيكون احسنلك تحذفها بدري عشان السرعة

او حتى انك تعمل العكس و تخلي البيانات تعيش بعد ما الدالة بتاعها انتهت

#44 - Dynamic Arrays

من الفوائد الرائعة للpointers هو استخدام ال `Pointer Arrays`

و ده مفيد لأن الArray العادية مكانتش بتسمح بتخزين عدد عناصر غير محدد أو باستخدام متغير

يعني مكانش ينفع نعمل كدا

```
int x;
cin >> x;
int Arr[x]; //Error
```

بس دلوقتي مع وجود البوينترز نقدر نعمل كدا

```
int n;
cin >> n;

int* arr = new int[n];
```

ده بيحجز اي عدد المستخدم عايزه

عشان تحذفها بتكتب `delete[]` :

```
delete[] arr;
```

ناخد دلوقتي مثال كامل عن اخذ عدد عناصر ليستة من المستخدم و ملئها بدرجات الطلاب

```
int num;
cout << "Enter total number of students: ";
cin >> num;

float * ptr;
// memory allocation of num number of floats
ptr = new float[num];

cout << "Enter grades of students." << endl;
for (int i = 0; i < num; i++) {
    cout << "Student" << i + 1 << ": ";
    cin >> *(ptr + i);
}

cout << "\nDisplaying grades of students." << endl;
for (int i = 0; i < num; i++) {
    cout << "Student" << i + 1 << ": " << *(ptr + i) << endl;
}
// ptr memory is released
delete[] ptr;
```

#45 - Stack vs Heap

أولا كل اللي هنتكلم عنه دلوقتي هيكون متخزن جوا الرام

هنتكلم دلوقتي علي اربع اجزاء في الميموري بتاعتك أو في الرام بتاعتك

Source Code

الجزد ده بيكون حجمه صغير عشان بطبيعة الحال التعاليم البرمجية بتكون صغيرة مقارنة بالحجم الفعلي للبيانات اللي البرنامج بيخزنها فا بالتالي الجزء ده مساحته صغيرة

Static / Global

المنطقة دي هي اللي بيتم تخزين فيها ال `Global variables` وال `static variables`

دي المتغيرات اللي بيفضل ليها Access طول فترة حياة البرنامج

عشان اقدر احافظ عليهم طول فترة حياة البرنامج بيتم وضعهم في المكان ده لوحدهم

Stack

ده بقا المكان اللي بيتخزن فيه معظم بياناتك , من الFunction وال `Local variables` وال `Pointers` وهكذا

بيكون حجمه متحدد قبل ما البرنامج يبدا , إزاي؟

البرنامج بيقرا المتغيرات و الدوال اللي انت عرفتها قبل ما يشتغل و علي اساسها بيحدد المساحة اللي انت محتاجها في الستاك و يديهالك

بيكون في حدود ال2 ميجا ولا حاجة

Heap

من بعد ما المترجم يديك المساحة اللي هو قررها للStack انت مش هيكون ليك الحق تحجز اكتر منهم خلاص (احيانا بتكون محتاج اكتر منهم) بس في طريقة في انك تاخد مساحة كمان و هي انك تعمل أكسيس علي الميموري اصلا باستخدام ال `Pointers` و دي اتكلمنا عليها قبل كده عشان البوينتر بيقدر يخش علي الداناميك موميري (Heap)

> **يعني الميموري متخزن علي الاستاك بس هو عبارة عن مفتاح للوصول للHeap لتخزين ما تريد من المساحة و التحكم فيها بشكل مفتوح**

ساعتها بيكون ليك باقيت المساحة و دي بتقدر بالجيجات (ليها ليميت برضو بس عالي)

يعني البوينتر عامل زي دفتر شيكات بتكتب فيه الشيك اللي انت محتاجه عشان تأكسيس مساحة اكبر من اللي معاك

هتفهم بعدين قد ايه الموضوع مفيد ومهم

4 Heap: any dynamic variables / objects / arrays

المكان ده مهم جدا و ده المكان اللي بتجيب منه مساحة زيادة ليك من بعد ما البرنامج يشتغل و ده بيتم بواسطة البوينتر اللي بيشتغل كأنه مفتاح علي الذاكرة

3 Stack: local variables / functions / pointers

هنا الكومبايلر بيشوف انت عايز مصروف قد ايه عشان تعرف تخزن متغيراتك و بيكون مصروفك مقدر بالميجات و مش بيتغير لما البرنامج يشتغل

2 Static / Global

هنا المكان اللي بيخزن فقط المتغيرات العامة اللي بتفضل عايشة طيلة حياة البرنامج

1 Source Code / Instructions

هنا المكان اللي بيخزن التعاليم البرمجية و بيكون حجمها صغير

Vectors Part II

#46 - Access Elements

تقدر تعمل أكسيس للعناصر بتاعت الفيكتور باستخدام `[] operation` كلعادة بس في كمان طريقة و هي استخدام الدالة `at`

```
vector<int> num{ 1, 2, 3, 4, 5 };

cout << "\n\n using .at(i) \n";
cout << "Element at Index 0: " << num.at(0) << endl; // 1
cout << "Element at Index 2: " << num.at(2) << endl; // 3
cout << "Element at Index 4: " << num.at(4) << endl; // 5

cout << "\n\n using [i]\n";
cout << "Element at Index 0: " << num[0] << endl; // 1
cout << "Element at Index 2: " << num[2] << endl; // 3
cout << "Element at Index 4: " << num[5] << endl; // 5
```

بتعمل نفس المهمة بالظبط بس ليها ميزة :

> لما تحاول توصل لعنصر ترتيبه مش موجود بيرفعلك ايرور علي عكس ال[ ] مش بترفعلك ايرور

#47 - Change Elements

عندك أكتر من طريقة لتغيير قيم العناصر , أولا انك تستخدم ال `Ranged loop` مع استخدام متغيرات المرجع `int &`

يستحسن لو هتقرا بس بدون تغيير يبقا تستخدم `const int&` بحيث لما تحاول تغيرها يرفعلك ايرور تغيير متغير ثابت

الطريقة التانية هي التغيير باستخدام `[]` او `at`

```
vector<int> num{ 1, 2, 3, 4, 5 };
cout << "Initial Vector: ";
for (const int& i : num) { // للقرائة فقط
    cout << i << " ";
}
cout << "\n\nUpdated Vector: ";
for ( int& i : num) { // للقرائة او التغيير
    i = 20; // -> تغيير جميع العناصر لـ20
    cout << i << " ";
}
num[1] = 40; // تغيير العنصر التاني
num.at(2) = 80; // التالت
num.at(4) = 90; // الخامس
cout << "\n\nUpdated Vector: ";
for (const int& i : num) { // للقرائة
    cout << i << " ";
}
```

```
Initial Vector: 1 2 3 4 5
Updated Vector: 20 20 20 20 20
Updated Vector: 20 40 80 20 90
```

#48 - Vector Iterators

الIterator ده عبارة عن اوبجيكت بيتحرك علي عناصر الVector عنصر عنصر عن طريق عناوينهم , بدل ما تستخدم الindex `v[0], v[1], v[2]` بتستخدمها لانها مهمة جدا بعدين

أولا كيفية التعريف

```
vector<int>::iterator iter;
```

> يعني : عرفلي `iterator` بيتحرك علي `vector` من نوع `int`

تمام دلوقتي ازاي نربطه ببداية الفيكتور بتاعنا؟ : باستخدام دالة `v.begin()` اللي بتجيب عنوان اول عنصر

```
iter = v.begin();
```

تمام , دلوقتي ازاي نحركه؟ : ببساطة بنزود واحد `iter++`

و بنقدر نجيب قيمة العنصر اللي هو واقف عليه باستخدام النجمة

```
iter++;
cout << "currant element is " << *iter << endl;
```

تعالى نطبق كل ده في مثال واحد :

```
vector<int> num{ 1, 2, 3, 4, 5 };

// declare iterator
vector<int>::iterator iter;

// use iterator with for loop
for (iter = num.begin(); iter != num.end();iter++) {
    cout << *iter << " ";
}
```

الفور لووب بتاعتنا عملت التالي :

1- اعطت قيمة للiterator بتاعنا تساوي عنوان اول عنصر

2- بعد كل خطوة بتزود الiterator بمقدار 1

3- لما الiterator يساوي عنوان أخر عنصر , اللوب بتنتهي

4- بيتم عمل اكسيس باستخدام `*iter` وليس `iter`

و كمان تقدر تغير قيمة العنصر باستخدام ال `*iter`

#49 - Try & Catch

لو حصل Error في البرنامج من انواع الايرورز اللي بتعمل كراش او تقفل البرنامج بتاعك ساعتها لو حصل خطأ واحد فقط أثناء عمل البرنامج , ساعتها بيقفل كله

عشان كده من الضروري يكون في حاجة تهاندل الموضوع ده ولو حصل خطأ متطفيش البرنامج بل وكمان تعطيك رسالة تعرفك ان في ايرور حصل

```
vector<int> num{ 1, 2, 3, 4, 5 };
try {
    cout << num.at(5); // -> السطر اللي شاكك انه ممكن يطلع ايرور
}
catch (...) {
    cout << "out of bound\n";
}
```

خذ بالاعتبار ان ده بيبطأ البرنامج شوية فا متستخدمهاش ع الفاضي و المليان بس طور اللوجيك بتاعك عشان الاخطاء متكترش

#50 - String Object: (Common Methods)

هناخد هنا شوية دوال علي الStrings من مكتبة `string`

أولا نعرف string

```
string S1= "My Name is Abdelrahman Mohye, I Love Programming.";
```

نبدأ بدالة `length` لإيجاد الطول

```
cout << S1.length() ; // 49
```

دالة `at()` اللي اخدناها قبل كدا

```
cout << S1.at(3) ; // N
```

دالة `append` اللي بتضيف لأخر النص

```
S1.append(" @ProgrammingAdvices");
cout << S1 << endl;
```

```
My Name is Abdelrahman Mohye, I Love Programming.
@ProgrammingAdvices
```

دالة `insert` للاضافة في مكان معين

```
S1.insert(7, " Ali ");
cout << S1 << endl;
```

```
My Name Ali  is Abdelrahman Mohye, I Love Programming.
@ProgrammingAdvices
```

دالة `substr` لتحديد جزء من المتغير

```
cout << S1.substr(16, 11) << endl; // من الموقع 16 -> قص 11 حرف
```

```
Abdelrahman
```

دالة `find` لايجاد مكان نص

```
cout << S1.find("Ali") << endl;
```

```
8
```

لو العنصر ده مش موجود , القيمة اللي بتطلع بتكون `S1.npos`

```
if (S1.find("ali") == S1.npos) {
    cout << "ali is not found";
}
```